

SOURCE CODE GUIDE

Frappier is completed project template with all features which are commonly used in projects.

Technologies ASP.NET Core 3.1, Entity Framework Core 3.1, jQuery, Bootstrap, SQL Server

Feature Overview

- Alerts
 - Custom User Management
 - Unit Of Work Using Entity Framework
 - Unit Of Work Using Dapper
 - Configure Menus by Roles
 - Custom Grids
 - Captcha
 - Cache Services
 - Dapper
 - Entity Framework Core
 - Create Pdf files
 - Create Notice
 - Capture Photo from WebCam
 - Cropping Images and storing
 - Create Notice
 - Download Files
 - Download Excel Reports
 - Exception Logging
 - FluentValidation
 - Globalization and Localization
 - HealthChecks
 - jQuery Datetime pickers & Bootstrap Datetime pickers
 - Jtable Grid
 - Loader
 - Modal popups
 - Notification
 - Ordering Menus
 - Upload Files to store in a folder
 - Upload Files to store in Database
-

NuGet Packages which are used in Project

- AspNetCore.HealthChecks.Redis
 - AspNetCore.HealthChecks.SqlServer
 - AspNetCore.HealthChecks.UI
 - AspNetCore.HealthChecks.UI.Client
 - AspNetCore.HealthChecks.UI.InMemory.Storage
 - AspNetCore.HealthChecks.UI.SqlServer.Storage
 - AutoMapper.Extensions.Microsoft.DependencyInjection
 - DinkToPdf
 - DNTCaptcha.Core
 - EPPlus
 - FluentValidation.AspNetCore
 - Microsoft.AspNetCore.Mvc.NewtonsoftJson
 - Microsoft.AspNetCore.Mvc.Razor.RuntimeCompilation
 - Microsoft.EntityFrameworkCore
 - Microsoft.EntityFrameworkCore.SqlServer
 - Microsoft.EntityFrameworkCore.Relational
 - Microsoft.Extensions.Caching.Redis
 - Microsoft.VisualStudio.Web.CodeGeneration.Design
 - SixLabors.ImageSharp
 - StackExchange.Exceptional.AspNetCore
 - X.PagedList.Mvc.Core
 - Microsoft.Data.SqlClient
 - Dapper
 - Newtonsoft.Json
 - System.Linq.Dynamic.Core
 - System.ComponentModel.Annotations
-

Getting Started

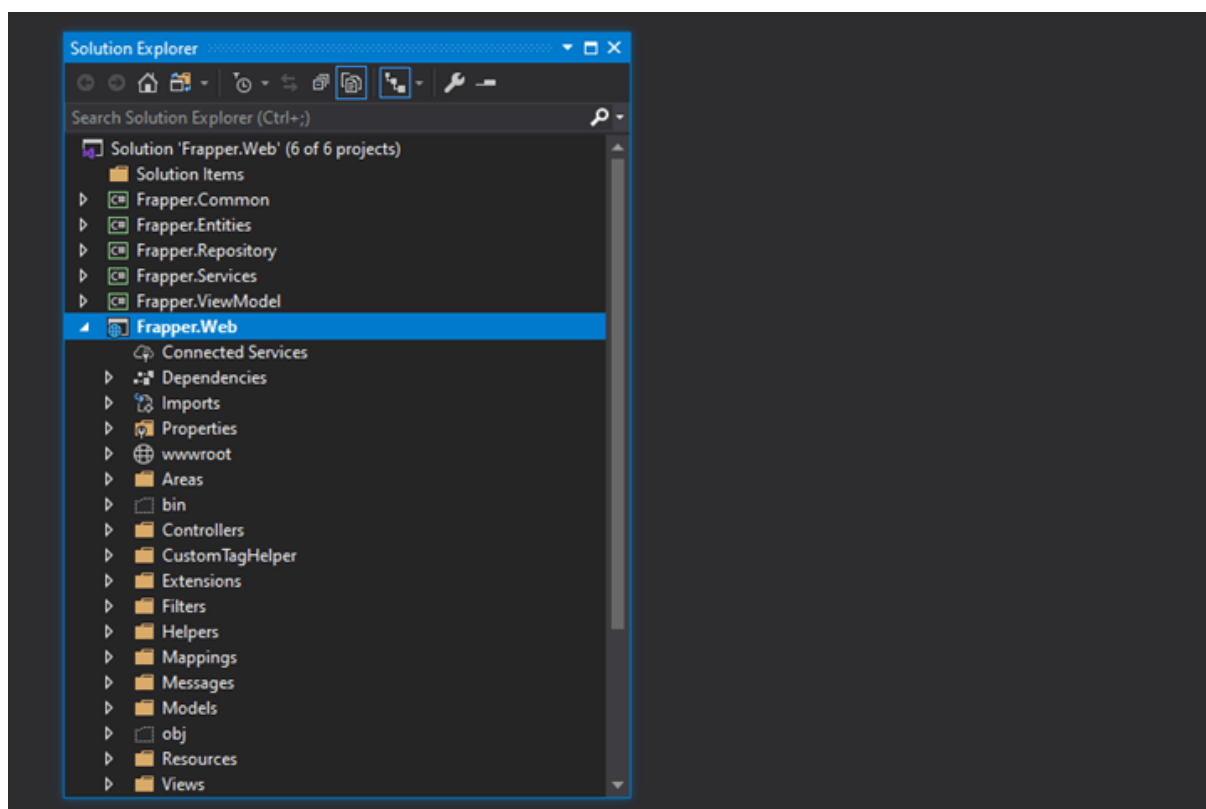
- Clone code from Github: git clone <https://github.com/saineshwar/Frapper>
- Open solution Frapper.Web.sln in Visual Studio 2019
- There are 2 Database Scripts FrapperDB (Main Database),FrapperAuditDB (Request Audit Database) Download Database Script
- appsettings.json file update DatabaseConnection (FrapperDB Database) , AuditDatabaseConnection (FrapperAuditDB Database)
- Run Database Script which is provided
- Make Changes in ConnectionStrings, ApplicationSettings, Exceptional, RedisServer in appsettings.json file
- Build project which will restore all NuGet Packages
- Final Step Run Project

Project Structure

Project Structure is simple to understand.

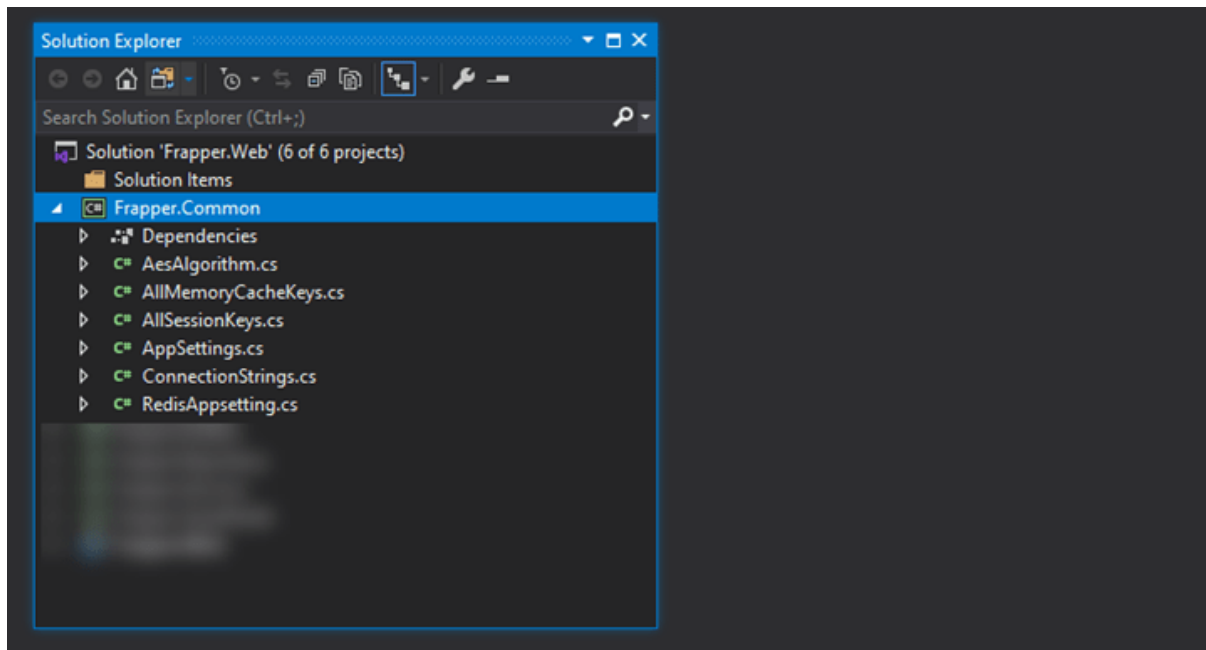
Frapper.Web

Frapper.Web which is Main ASP.NET Core Project Template.



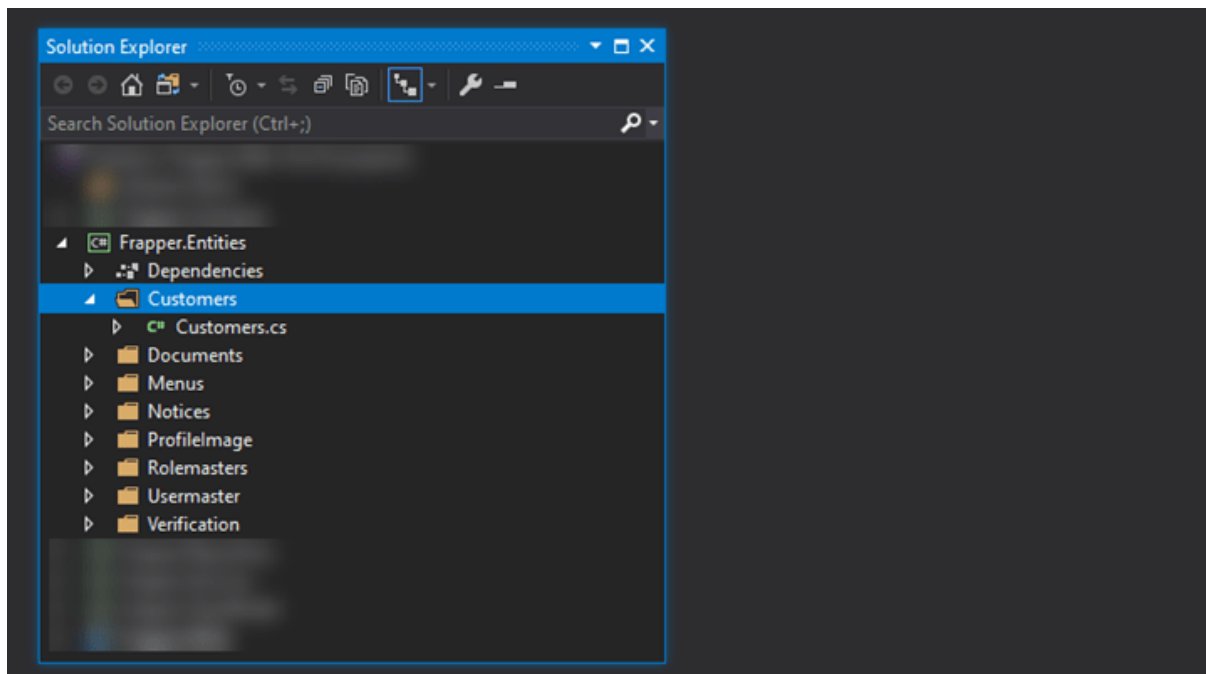
Frapper.Common

Frapper.Common this project contains all common libraries which are used in a project such as Session keys, AppSettings, ConnectionStrings, Algorithms.



Frapper.Entities

Frapper.Entities this project contains all Entities.

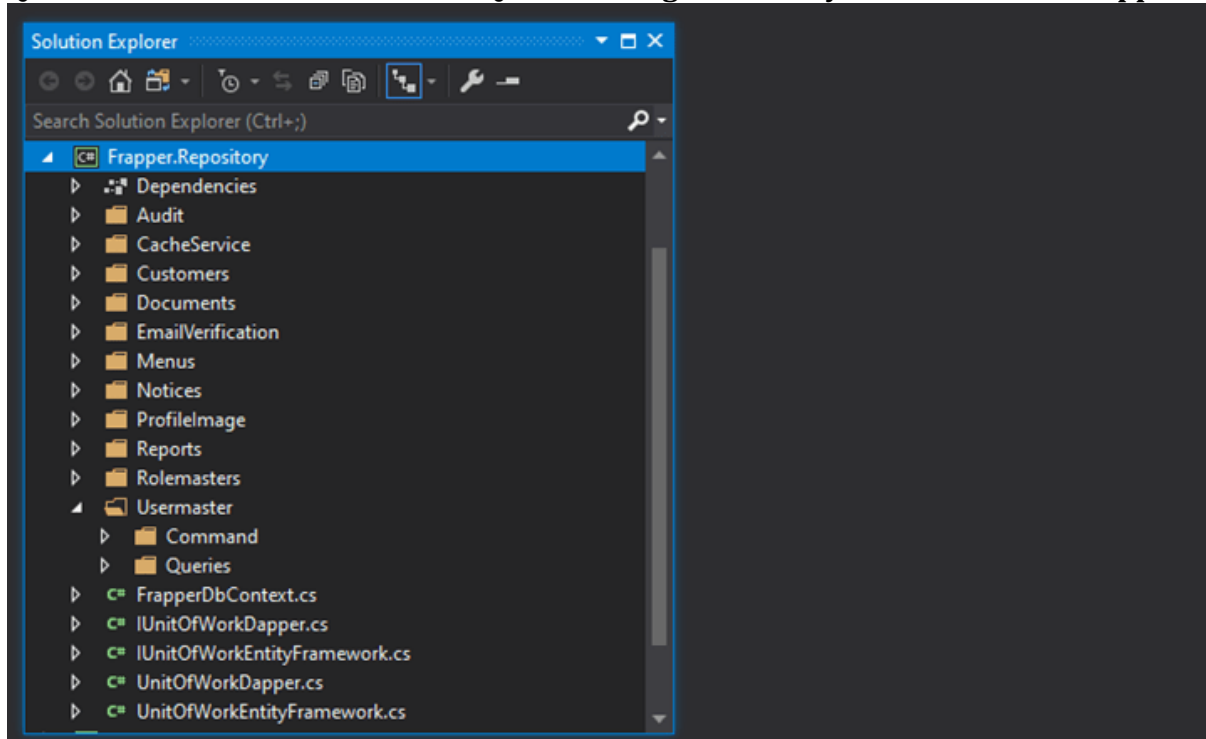


Frappor.Repository

Frappor.Repository this project contains all Class which access database. In this part we have separated Responsibility into two different part reading and writing.

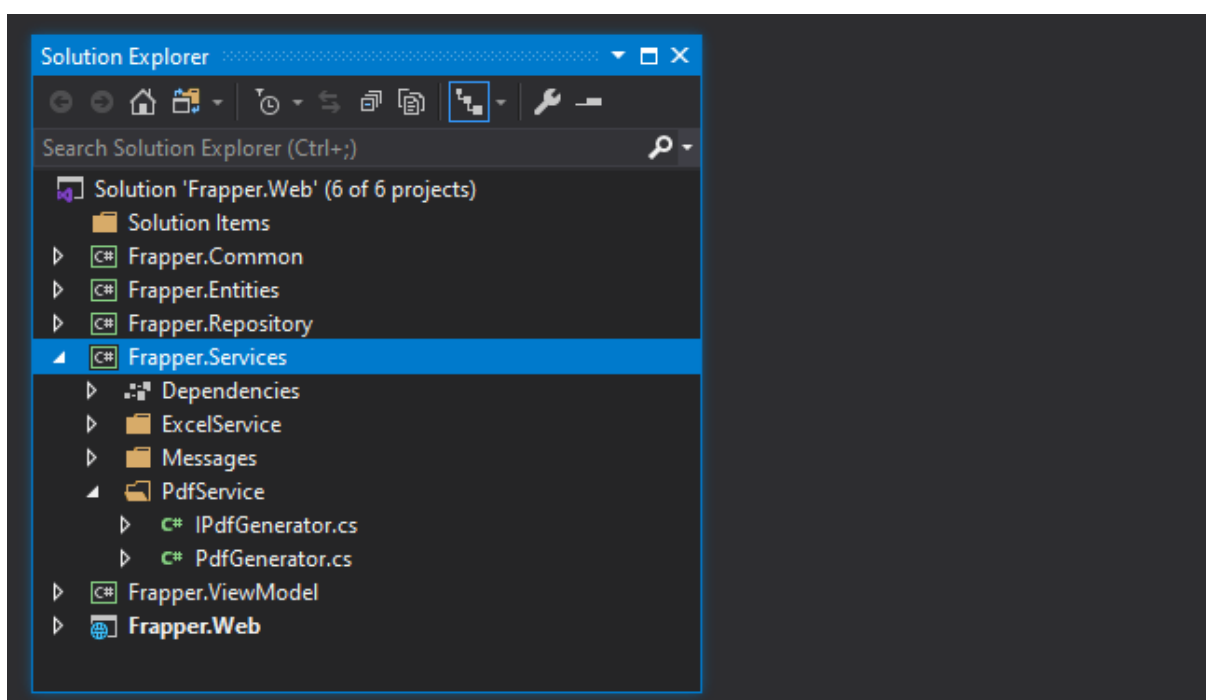
Command Folder contains all Commands such as Insert, Update, Delete using both Entity Framework and Dapper.

Queries Folder contains all Select Queries Using both Entity Framework and Dapper.



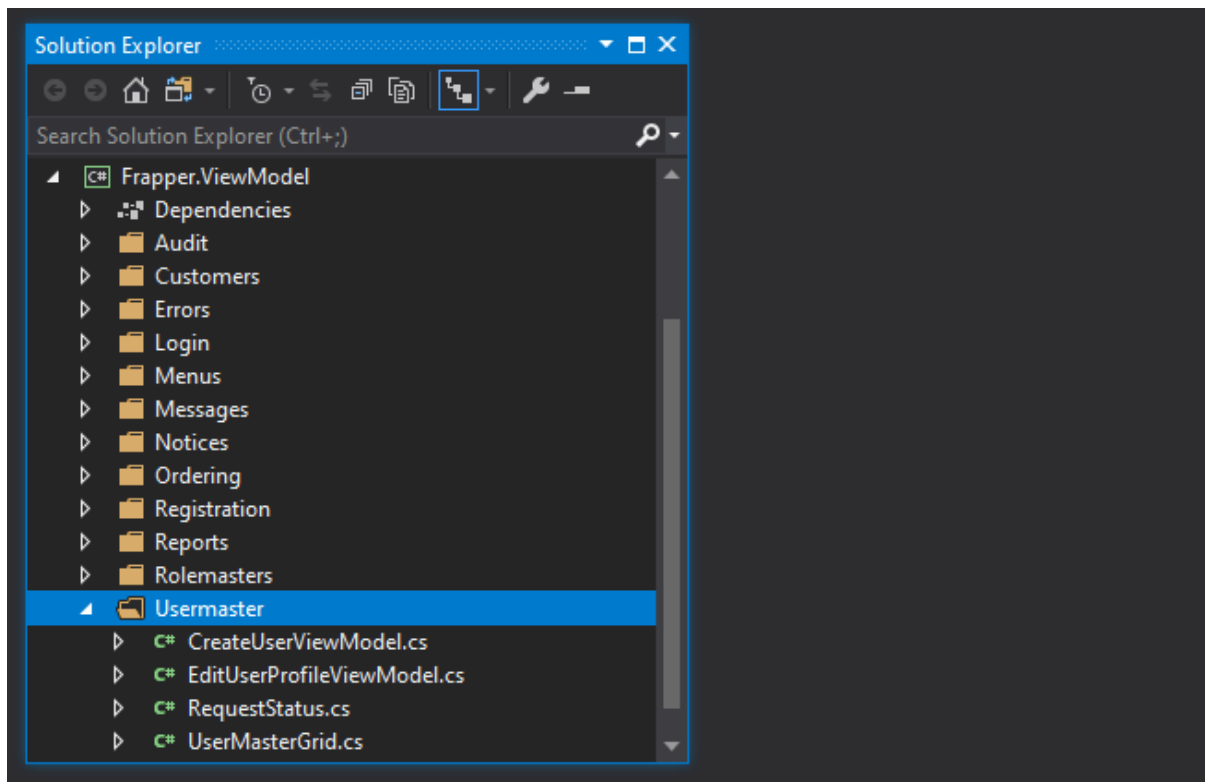
Frappor.Services

Frappor.Services this project is used to write business logic and also common services such as generating Excel, PDF, Sending Emails. Frappor.Services project has reference to with Frappor.Repository project which means it can access the database directly and Frappor.Services project can be accessed in Frappor.Web project.



Frappier.ViewModel

Frappier.ViewModel this project contains all ViewModel used in project.

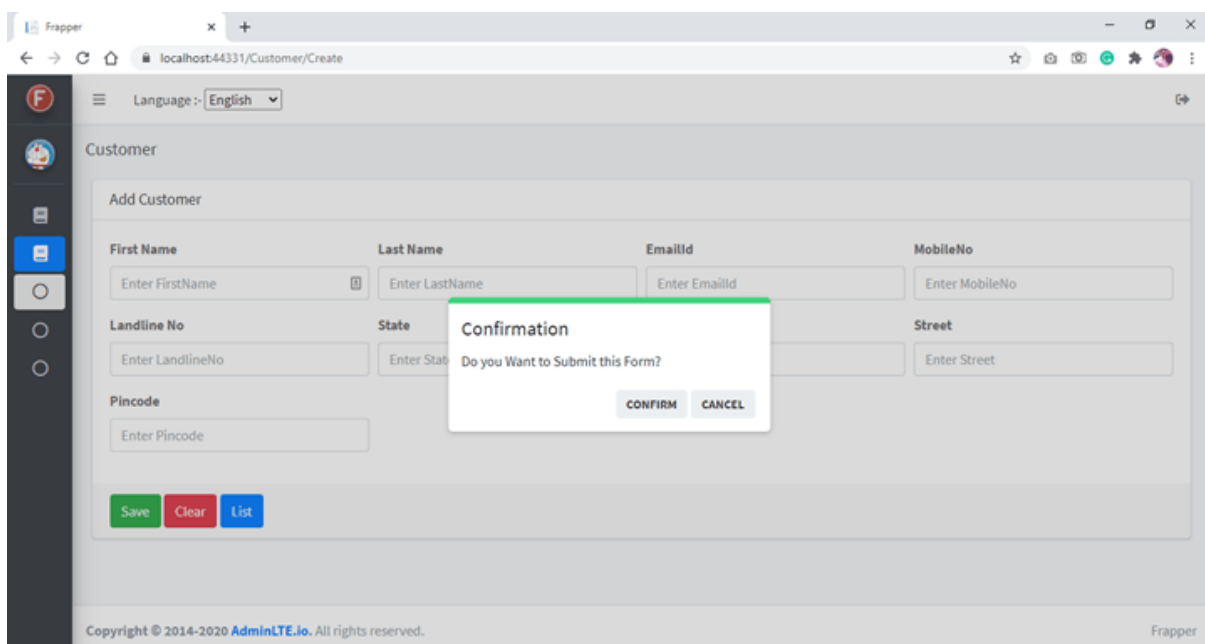


Till now we have seen Project structure of Frappier project now let's Move Forwards and see all features.

Features

Alerts

In this project we have used jQuery Confirm Alerts.

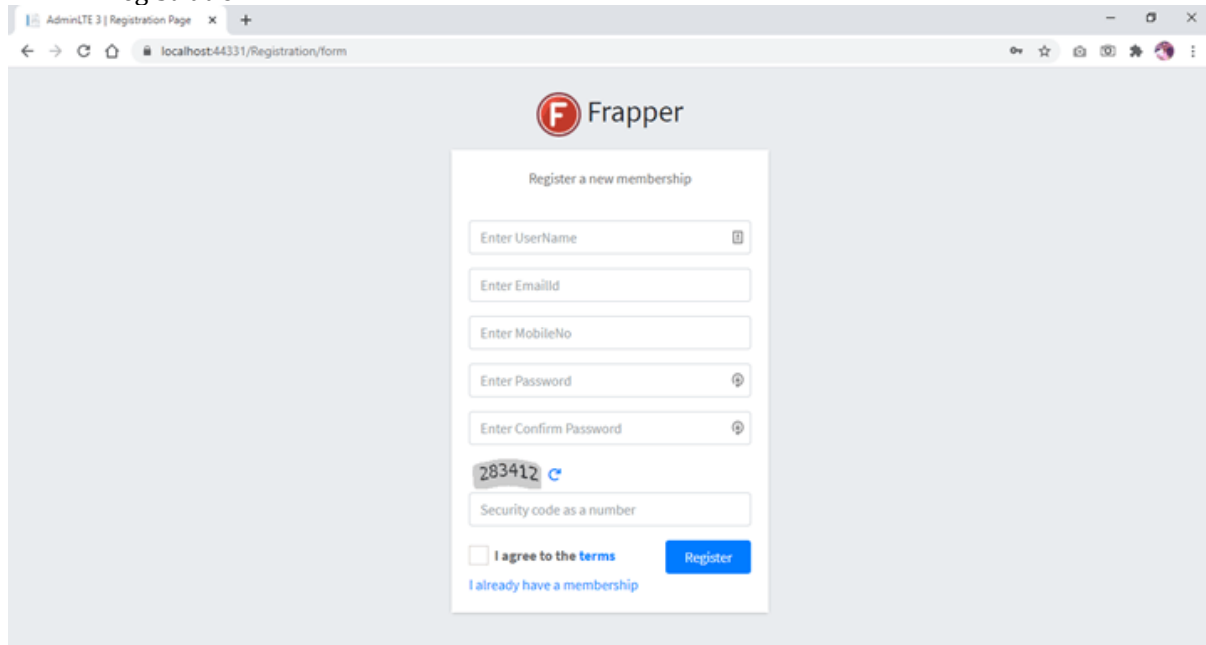


Custom User Management

Custom User Management which contains Login, Registration, Forgot Password, Change Password. Disabling and enabling users.

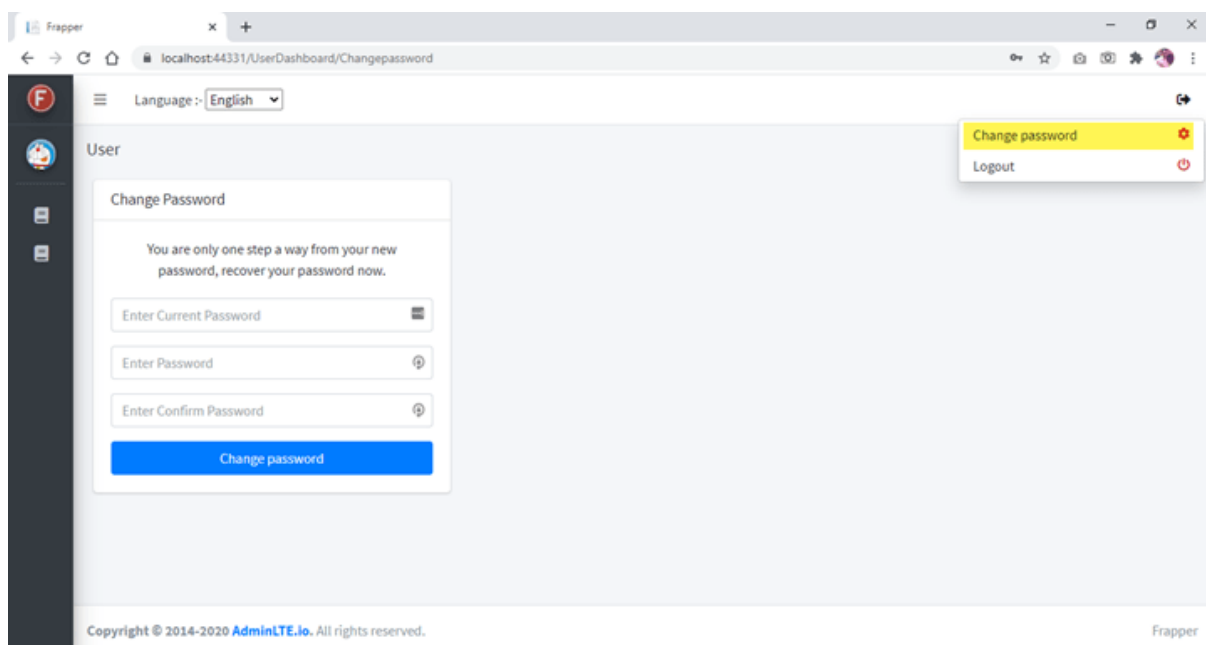
All CRUD operations are done using the Repository Pattern and Unit of Work Pattern. Also, we have separated Responsibility into two different part reading (queries) and writing (commands).

- Registration



The screenshot shows the 'Registration Page' of the Frapper application. The browser address bar indicates the URL is 'localhost:44331/Registration/form'. The page features the Frapper logo at the top center. Below the logo, the heading 'Register a new membership' is displayed. The registration form includes the following fields: 'Enter UserName', 'Enter EmailId', 'Enter MobileNo', 'Enter Password', 'Enter Confirm Password', and a security code field showing '283412' with a refresh icon. Below the security code field is a checkbox labeled 'I agree to the terms' and a link 'I already have a membership'. A blue 'Register' button is positioned to the right of the checkbox.

- Change password



The screenshot shows the 'Change Password' page of the Frapper application. The browser address bar indicates the URL is 'localhost:44331/UserDashboard/Changepassword'. The page has a dark sidebar on the left with the Frapper logo and a 'User' profile section. The main content area is titled 'Change Password' and includes the message: 'You are only one step a way from your new password, recover your password now.' The form contains three fields: 'Enter Current Password', 'Enter Password', and 'Enter Confirm Password'. A blue 'Change password' button is at the bottom of the form. In the top right corner, a user menu is open, showing 'Change password' (highlighted in yellow) and 'Logout' options. The footer of the page contains the copyright notice: 'Copyright © 2014-2020 AdminLTE.io. All rights reserved.' and the Frapper logo.

- Forgot Password

AdminLTE 3 | Forgot Password

localhost:44331/Portal/ForgotPassword

F Frapper

You forgot your password? Here you can easily retrieve a new password.

Username

454212

Enter Security code

[Request new password](#)

[Login](#)

[Register a new membership](#)

- Email Sent with Reset Password Link

AdminLTE 3 | Forgot Password

localhost:44331/Portal/ForgotPassword

F Frapper

Message! An email has been sent to the address you have registered. Please follow the link in the email to complete your password reset request.

You forgot your password? Here you can easily retrieve a new password.

Username

808135

Enter Security code

[Request new password](#)

[Login](#)

[Register a new membership](#)

Email Reset Link

Welcome to Frapper Inbox x

Frappier
to me

Welcome
Dear demodemodemo
Please click the following link to reset your password.
Reset password link : [Link](#)
If the link does not work, copy and paste the URL into a new browser window. The URL will expire in 24 hours for security reasons.
Best regards, Frapper

Unit Of Work Using Entity Framework

```

UnitOfWorkEntityFramework.cs
Frappier.Repository
1
2
3 using ...
10
11 namespace Frappier.Repository
12 {
13     28 references
14     public interface IUnitOfWorkEntityFramework
15     {
16         4 references
17         IRoleCommand RoleCommand { get; }
18         9 references
19         IUserMasterCommand UserMasterCommand { get; }
20         5 references
21         IAssignedRolesCommand AssignedRolesCommand { get; }
22         4 references
23         IUserTokensCommand UserTokensCommand { get; }
24         6 references
25         IMenuCategoryCommand MenuCategoryCommand { get; }
26         6 references
27         IMenuMasterCommand MenuMasterCommand { get; }
28         6 references
29         ISubMenuMasterCommand SubMenuMasterCommand { get; }
30         7 references
31         IVerificationCommand VerificationCommand { get; }
32         4 references
33         IProfileImageCommand ProfileImageCommand { get; }
34         6 references
35         INoticeCommand NoticeCommand { get; }
36         5 references
37         INoticeDetailsCommand NoticeDetailsCommand { get; }
38         3 references
39         IDocumentCommand DocumentCommand { get; }
40         34 references
41         bool Commit();
42         1 reference
43         void Dispose();
44     }
45 }
100 % No issues found

```

Unit Of Work Using Dapper

```

UnitOfWorkDapper.cs
Frappier.Repository
1 using Frappier.Repository.Audit.Command;
2 using Frappier.Repository.Customers.Command;
3 using Frappier.Repository.Menus.Command;
4
5 namespace Frappier.Repository
6 {
7     6 references
8     public interface IUnitOfWorkDapper
9     {
10         2 references
11         ICustomersCommand CustomersCommand { get; }
12         4 references
13         IOrderingCommand OrderingCommand { get; }
14         4 references
15         bool Commit();
16     }
17 }

```

Configure Menus by Roles

Frappier

localhost:44331/Administration/MenuMaster/Create

Language: English

Menu

Add

Role: ---Select---

Menu Category: ---Select---

Area: Enter Area

ControllerName: Enter Controller Name

ActionMethod: Enter Action Method

Menu Name: Enter Menu Name

☒ Active

[Save](#) [Clear](#) [List](#)

Copyright © 2014-2020 AdminLTE.io. All rights reserved. Frappier

Custom Grids

Frappier

localhost:44331/Administration/MenuMaster/Create

Language: Japanese

Customer

Search

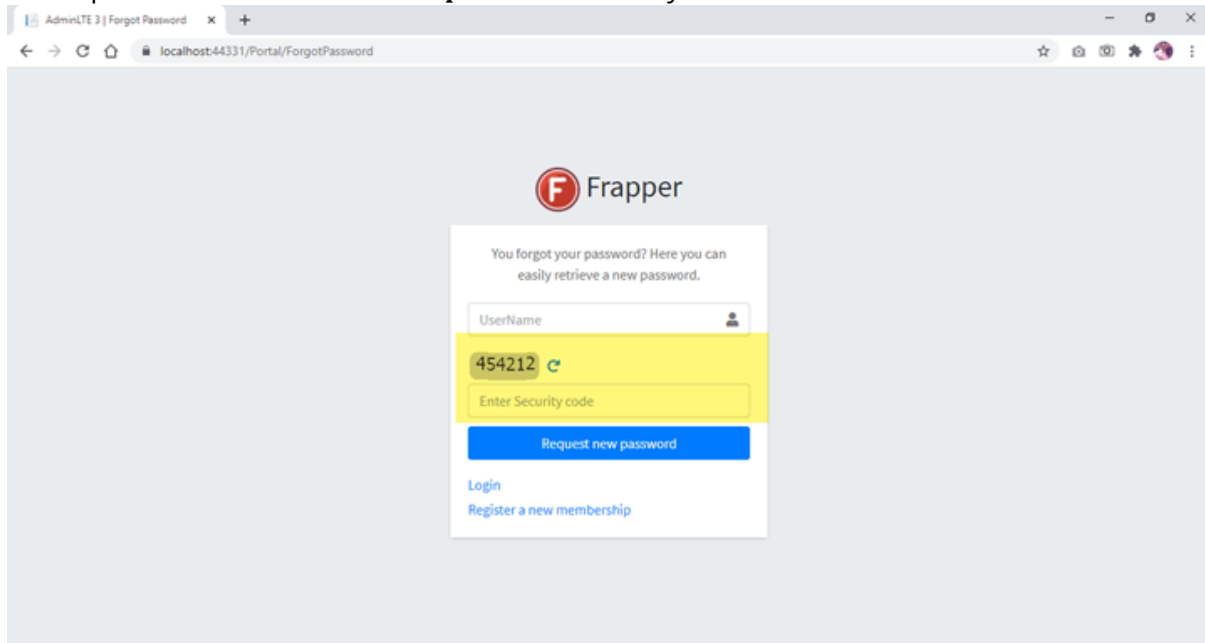
[Search](#) [Clear](#) [Add Customer](#)

ファーストネーム	苗字	電子メールアドレス	携帯電話番号	状態	市
jack	jack	jack@frapper.com	9879879877	Maharashtra	Mumbai
R	R	rick@frapper.com	9888888888	M	M
R	R	rick@frapper.com	9888888888	M	M
R	R	rick@frapper.com	9888888888	M	M
jess	mick	jmick@frapper.com	9852004242	Mumbai	M
Nishan	roy	nroy@frapper.com	9879879879	Maha	M
Shashank	Chawla	ShashankChawla@Frappier.com	975555388	Maharashtra	Mumbai
Varun	Chakrabarti	VarunChakrabarti@Frappier.com	975555388	Maharashtra	Mumbai
Krishna	Chowdhury	KrishnaChowdhury@Frappier.com	975555388	Maharashtra	Mumbai
Nishant	Dara	NishantDara@Frappier.com	975555388	Maharashtra	Mumbai

Copyright © 2014-2020 AdminLTE.io. All rights reserved. Frappier

Captcha

For Captcha we have used **DNTCaptcha.Core** Library.

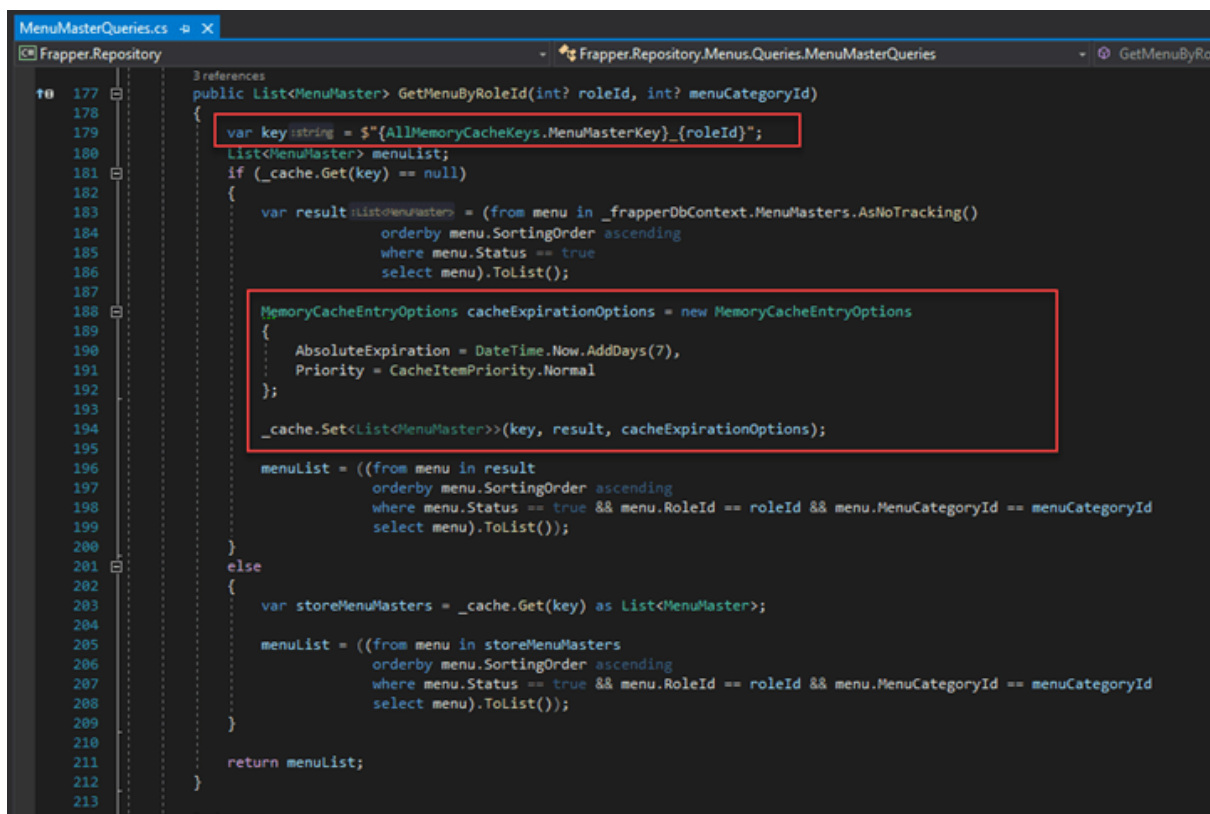


Cache Services

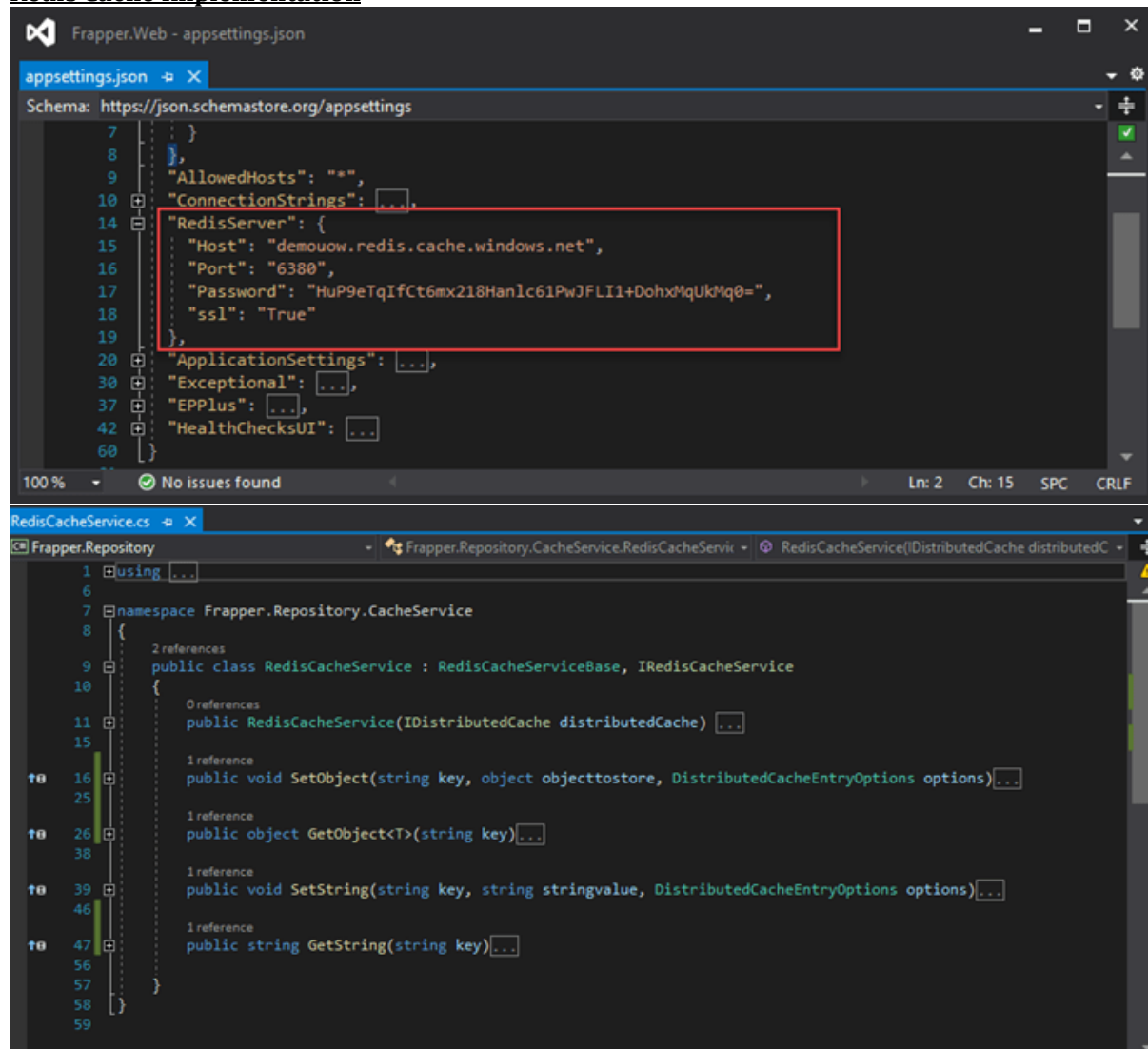
In this project we are using 2 caching services

1. Memory Cache
2. Redis

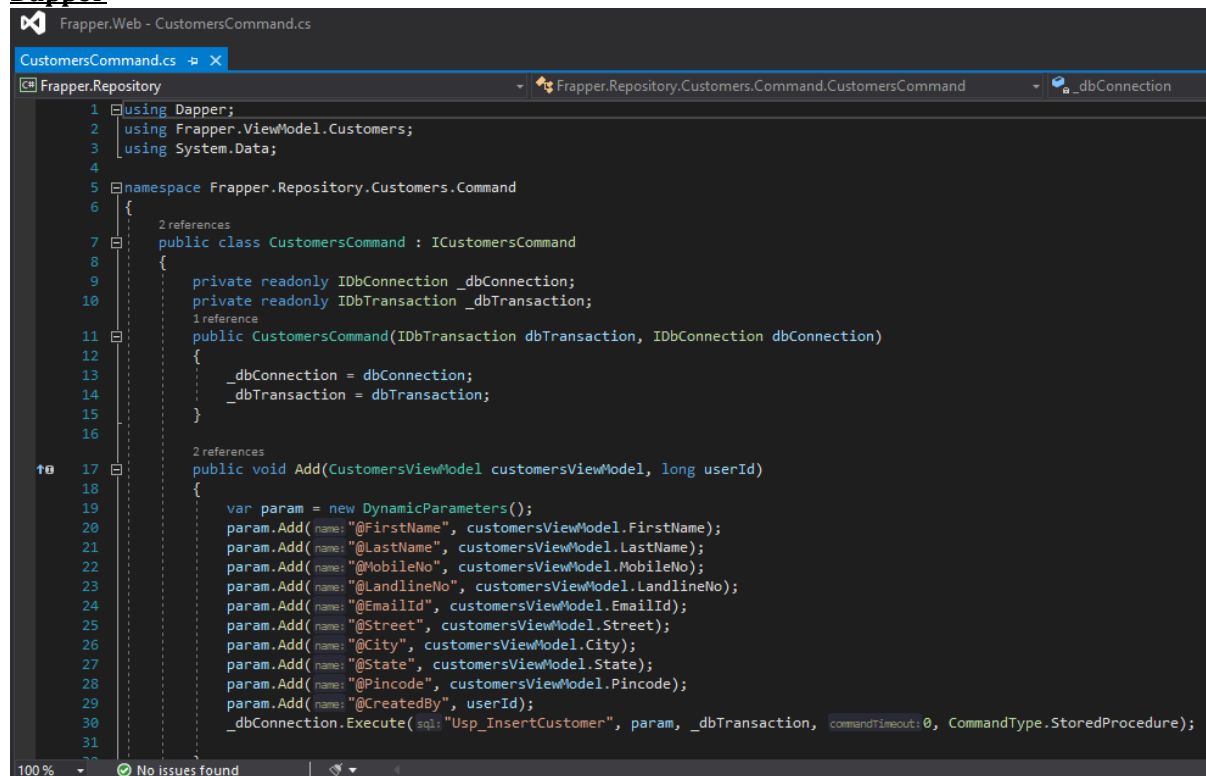
Memory Cache Implementation



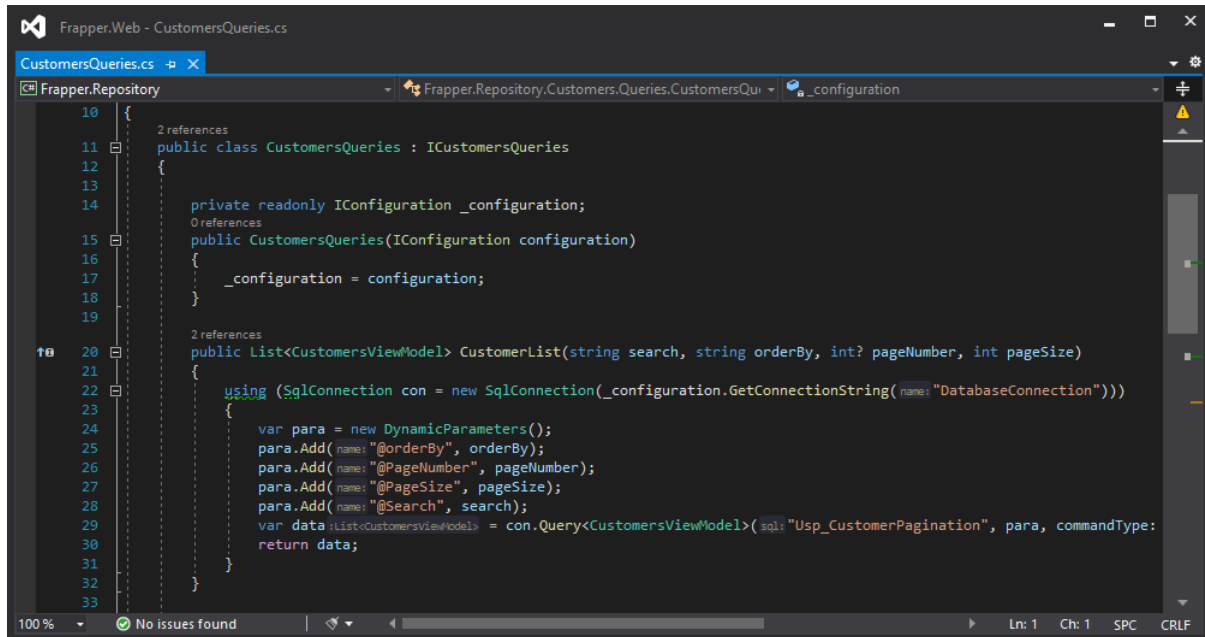
Redis Cache Implementation



Dapper



Dapper

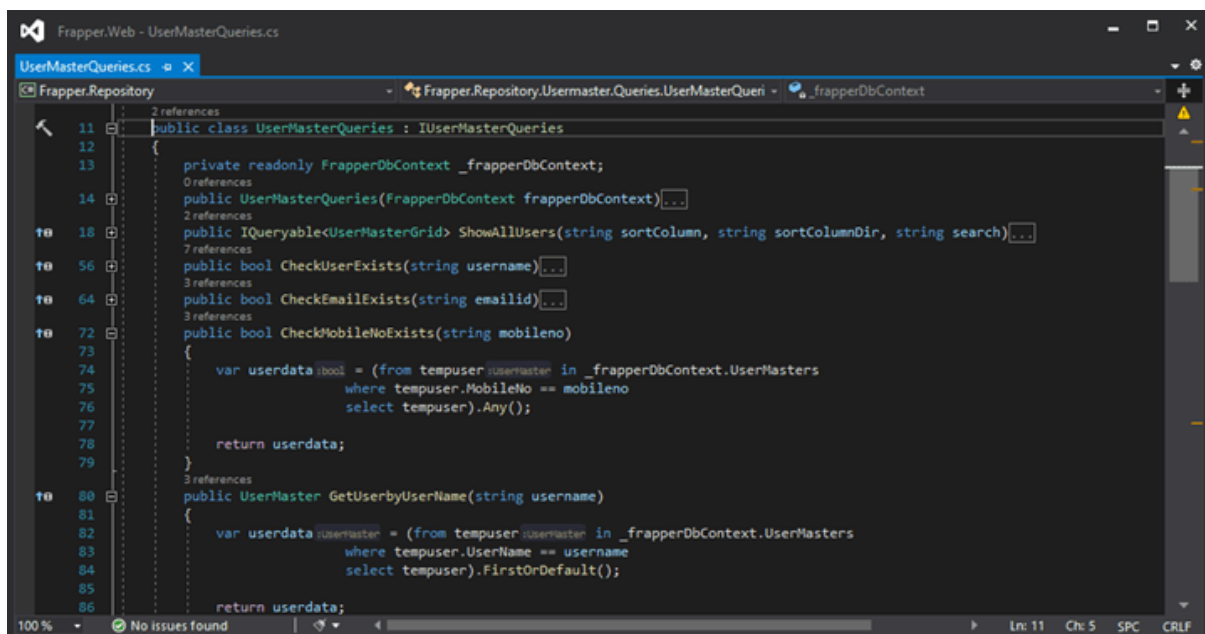


```

10  {
11      2 references
12      public class CustomersQueries : ICustomersQueries
13      {
14          private readonly IConfiguration _configuration;
15          0 references
16          public CustomersQueries(IConfiguration configuration)
17          {
18              _configuration = configuration;
19          }
20          2 references
21          public List<CustomersViewModel> CustomerList(string search, string orderBy, int? pageNumber, int pageSize)
22          {
23              using (SqlConnection con = new SqlConnection(_configuration.GetConnectionString( name: "DatabaseConnection")))
24              {
25                  var para = new DynamicParameters();
26                  para.Add(name: "@orderBy", orderBy);
27                  para.Add(name: "@PageNumber", pageNumber);
28                  para.Add(name: "@PageSize", pageSize);
29                  para.Add(name: "@Search", search);
30                  var data :List<CustomersViewModel> = con.Query<CustomersViewModel>(sql: "Usp_CustomerPagination", para, commandType:
31                  return data;
32              }
33      }

```

Entity Framework Core



```

11  2 references
12  public class UserMasterQueries : IUserMasterQueries
13  {
14      private readonly FrapperDbContext _frapperDbContext;
15      0 references
16      public UserMasterQueries(FrapperDbContext frapperDbContext) ...
17      2 references
18      public IQueryable<UserMasterGrid> ShowAllUsers(string sortColumn, string sortColumnDir, string search) ...
19      7 references
20      public bool CheckUserExists(string username) ...
21      3 references
22      public bool CheckEmailExists(string emailid) ...
23      3 references
24      public bool CheckMobileNoExists(string mobilenno)
25      {
26          var userdata :bool = (from tempuser :UserMaster in _frapperDbContext.UserMasters
27                               where tempuser.MobileNo == mobilenno
28                               select tempuser).Any();
29      }
30      return userdata;
31  }
32      3 references
33      public UserMaster GetUserByUserName(string username)
34      {
35          var userdata :UserMaster = (from tempuser :UserMaster in _frapperDbContext.UserMasters
36                                     where tempuser.UserName == username
37                                     select tempuser).FirstOrDefault();
38      }
39      return userdata;
40  }

```

Create Pdf files

We are using this **DinkToPdfLibrary** for creating pdf.

The screenshot shows the development process of a PDF generator. The top part is a Visual Studio Code editor window titled 'Frappier.Web' showing the 'PdfGenerator.cs' file. The code defines a 'PdfGenerator' class that takes an 'IConverter' as a dependency. It implements a 'DownloadInvoicepdf()' method that reads an HTML template, replaces placeholders with data, and uses the 'DinkToPdfLibrary' to convert the HTML to a PDF file. The bottom part is a web browser window showing the generated PDF invoice. The invoice is for 'John Doe' and includes a table of items with columns for Qty, Product, Serial #, Description, and Subtotal. The items listed are 'Call of Duty', 'Need for Speed IV', 'Monsters DVD', and 'Grown Ups Blue Ray'. The total amount is \$25.99. The browser also shows payment methods like Visa, Mastercard, and PayPal.

```

12 {
13     private readonly IConverter _converter;
14     public PdfGenerator(IConverter converter)
15     {
16         _converter = converter;
17     }
18     public FileResult DownloadInvoicepdf()
19     {
20         var basePath = new PhysicalFileProvider(Path.Combine(Directory.GetCurrentDirectory(), "wwwroot", "Templates")).Root + "invoice.html";
21         string strMailBody;
22         using (StreamReader objectStreamReader = new StreamReader(basePath))
23         {
24             strMailBody = objectStreamReader.ReadToEnd();
25             strMailBody = strMailBody.Replace(placeholder: "###CompanyName", newvalue: "Frappier");
26             strMailBody = strMailBody.Replace(placeholder: "###Date", newvalue: DateTime.Now.ToString());
27         }
28         var globalSettings = new GlobalSettings();
29         var objectSettings = new ObjectSettings()
30         {
31             PagesCount = true,
32             HtmlContent = strMailBody
33         };
34         var pdf = new HtmlToPdfDocument()
35         {
36             GlobalSettings = globalSettings,
37             Objects = { objectSettings }
38         };
39         var fileStream = _converter.Convert(pdf);
40         FileResult fileResult = new FileContentResult(file, contentType: "application/pdf");
41         fileResult.FileName = "Invoice_" + DateTime.UtcNow.ToString(CultureInfo.InvariantCulture) + ".pdf";
42         return fileResult;
43     }
44 }

```

The generated PDF invoice is displayed in a web browser. It includes the following information:

- To:** John Doe, 795 Folsom Ave, Suite 600, San Francisco, CA 94107, Phone: (555) 539-1037, Email: john.doe@example.com
- Invoice #007612**
- Order ID:** 4F358J
- Payment Due:** 2/22/2014
- Account:** 968-34567

Qty	Product	Serial #	Description	Subtotal
1	Call of Duty	455-981-221	El snort testosterone trophy driving gloves handsome	\$64.50
1	Need for Speed IV	247-925-726	Wes Anderson umami biodiesel	\$50.00
1	Monsters DVD	735-845-642	Terry Richardson helvetica tousled street art master	\$10.70
1	Grown Ups Blue Ray	422-568-642	Tousled lomo letterpress	\$25.99

Payment Methods: VISA, Mastercard, American Express, PayPal

Etsy doostang zoodles disqus groupon greplin oooj voxy zoodles, weebly ning heekya handango imeem plugg dopplr jibjab, movity jajah pickers sifted edmodo ifttt zimbra.

Create Notice

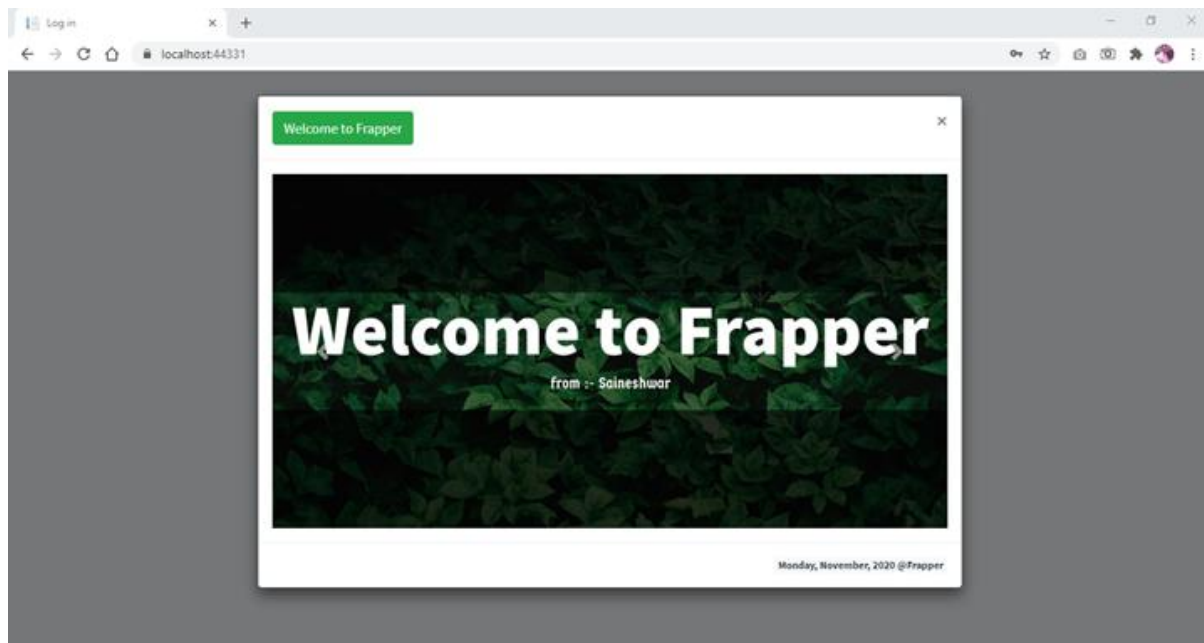
In creating Notice form we have used **Date Range Picker** and **CKEditor**.

The screenshot shows the 'Add Notice' form in the Frapper application. The form has the following fields and components:

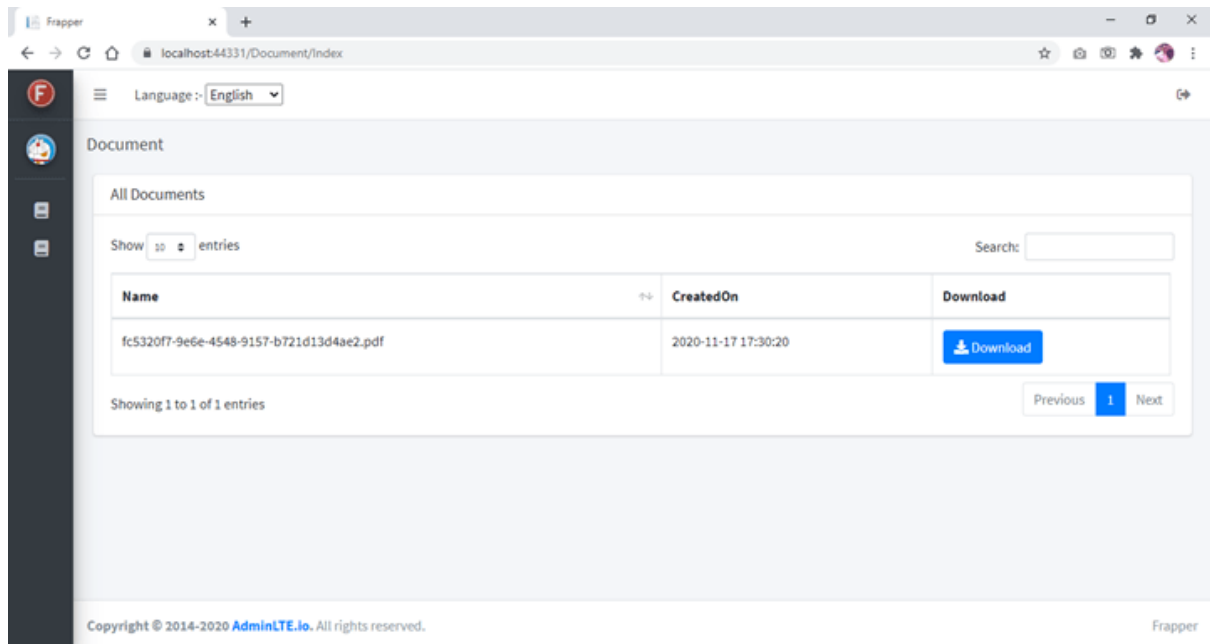
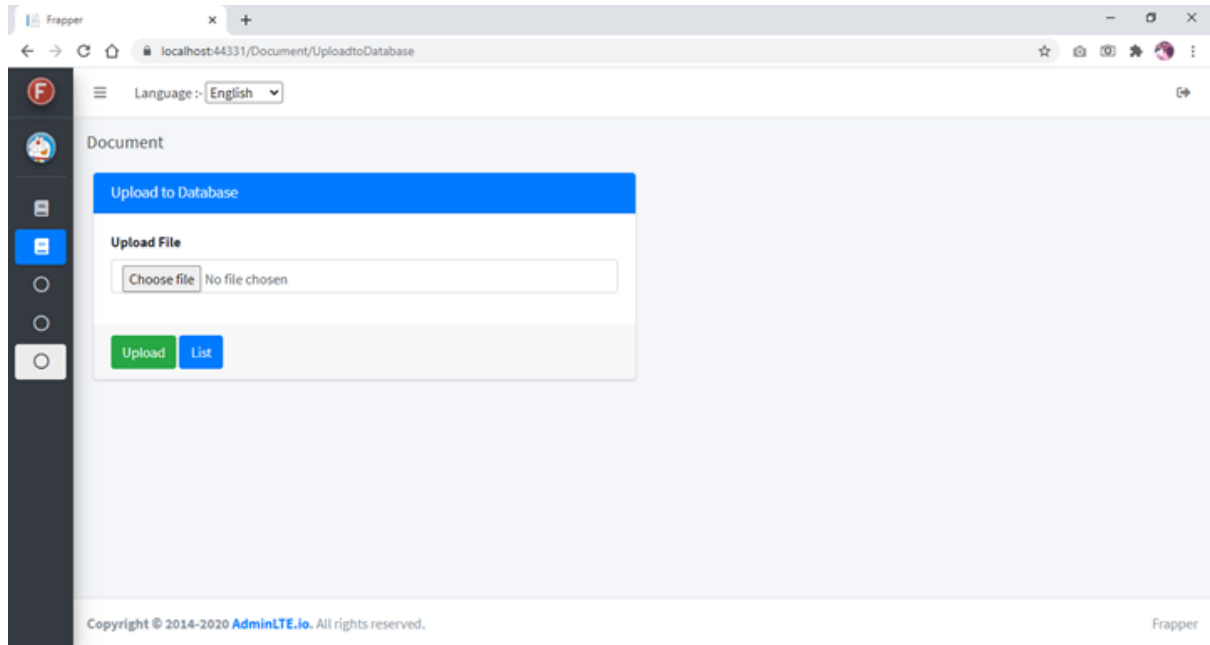
- Language:** English (dropdown)
- NoticeTitle:** Enter FullName
- NoticeStart:** 2020-11-18 19:23:24
- NoticeEnd:** Enter Notice End Date
- NoticeBody:** A CKEditor with a toolbar containing bold, italic, strikethrough, link, unlink, bulleted list, numbered list, indent, outdent, undo, redo, and other formatting options.
- Date Range Picker:** A calendar for November 2020 is open, showing the date 18 selected. The time is set to 7:23 PM.
- Buttons:** Save (green), Clear (red), List (blue)
- Footer:** Copyright © 2014-2020 AdminLTE.io. All rights reserved. Frapper

Rendering Notice

For rendering Notice we have used Bootstrap Modal pop.

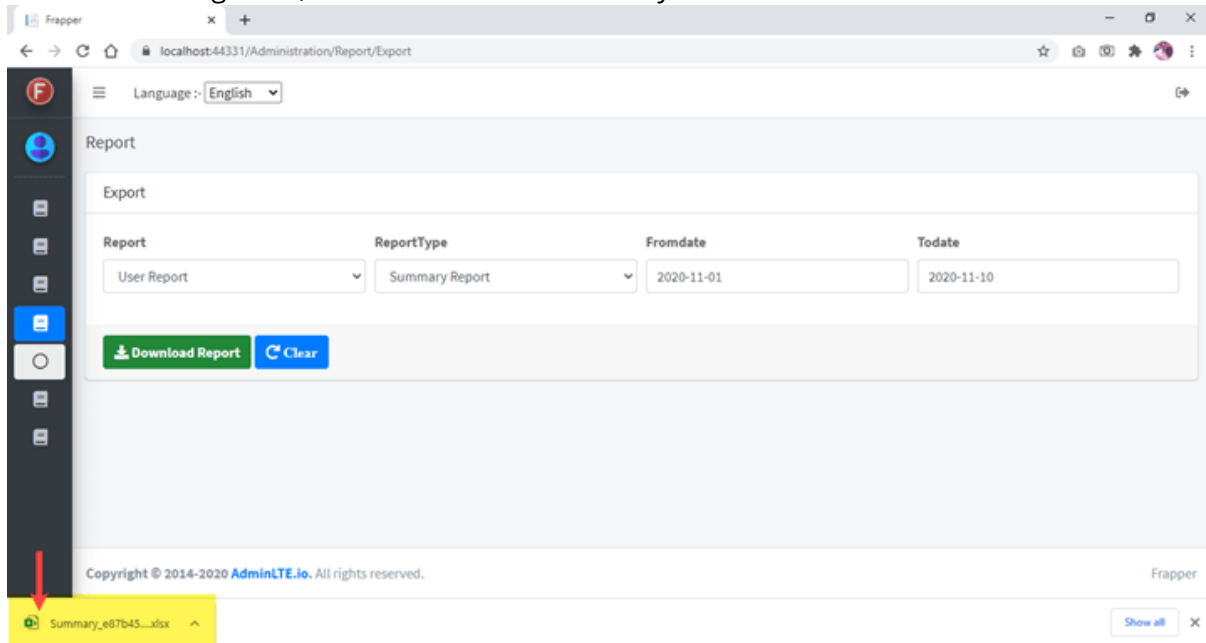


Download Files



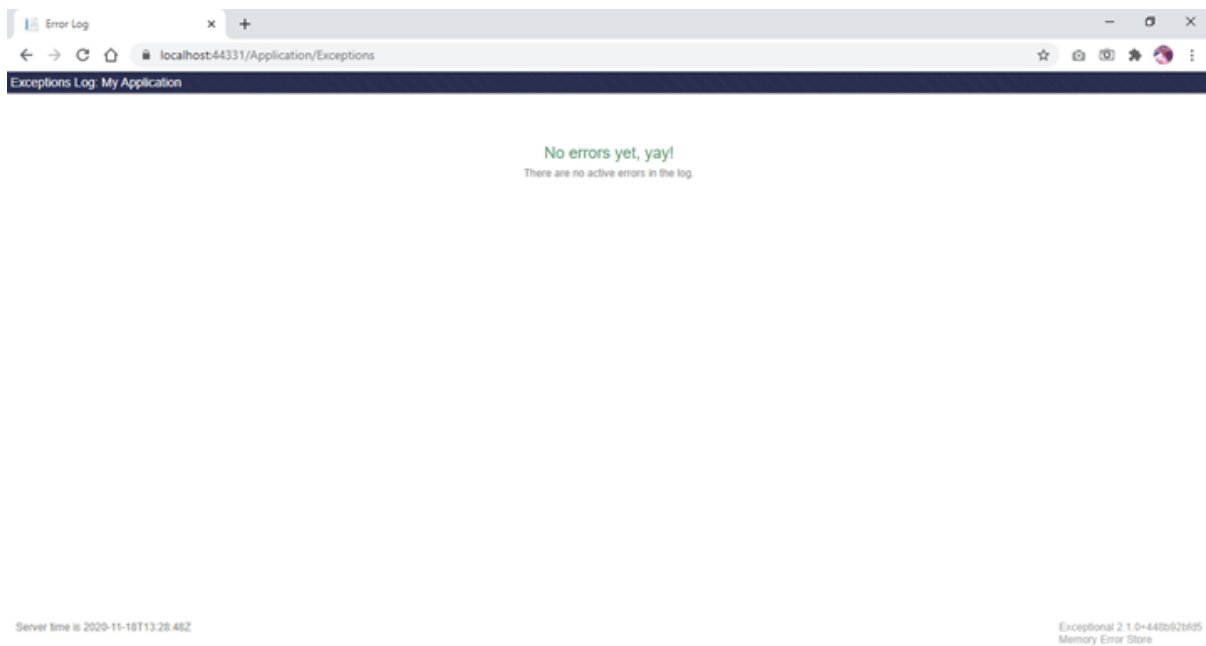
Download Excel Reports

For downloading Excel, we have used **EPLUS** library.



Exception Logging

For Logging exception, we have to use `StackExchange.Exceptional.AspNetCore`.



FluentValidation

FluentValidation validation library for building strongly-typed validation rules.

```

1 using FluentValidation;
2 using Frapper.ViewModel.Customers;
3
4 namespace Frapper.Web.Validators
5 {
6     2 references
7     public class CustomerValidator : AbstractValidator<CustomersViewModel>
8     {
9         0 references
10        public CustomerValidator()
11        {
12            RuleFor( @expression: x:CustomersViewModel => x.FirstName).NotEmpty()
13                .WithMessage("Please Enter 'FirstName'")
14                .Matches( @expression: @"^[a-zA-Z ]*$").WithMessage("Enter Valid First Name")
15                .MinimumLength(3).WithMessage("minimum length of 3 characters are allowed")
16                .MaximumLength(50).WithMessage("maximum length of 50 charaters is allowed");
17
18            RuleFor( @expression: x:CustomersViewModel => x.LastName).NotEmpty()
19                .WithMessage("Please Enter 'LastName'")
20                .Matches( @expression: @"^[a-zA-Z ]*$").WithMessage("Enter Valid Last Name")
21                .MinimumLength(3).WithMessage("minimum length of 3 characters are allowed")
22                .MaximumLength(50).WithMessage("maximum length of 50 charaters is allowed");
23
24            RuleFor( @expression: x:CustomersViewModel => x.MobileNo).NotEmpty()
25                .WithMessage("Please Enter 'MobileNo'")
26                .Matches( @expression: "^[7-9][0-9]{9}$").WithMessage("Mobile No. code should start with 7,8,9");
27
28            RuleFor( @expression: x:CustomersViewModel => x.LandlineNo).NotEmpty()
29                .WithMessage("Please Enter 'LandlineNo'")
30                .Matches( @expression: @"^\d{10}$").WithMessage("Invalid LandlineNo");
31
32            RuleFor( @expression: x:CustomersViewModel => x.EmailId) // IRuleBuilderInitial<CustomersViewModel, >
33                .NotEmpty().WithMessage("Please Enter 'EmailId'")

```

Globalization and Localization

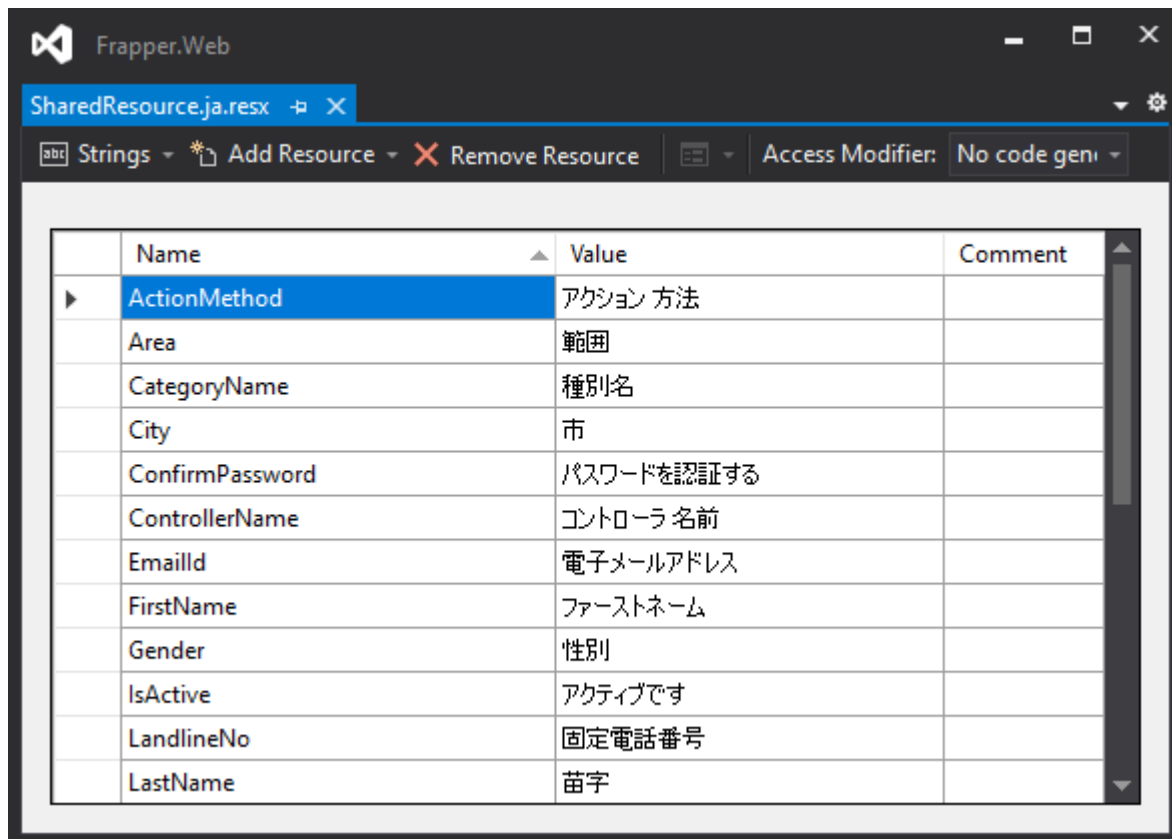
We have used 5 languages for demo. For that we have created 5 resource files.

Copyright © 2014-2020 AdminLTE.io. All rights reserved. Frapper

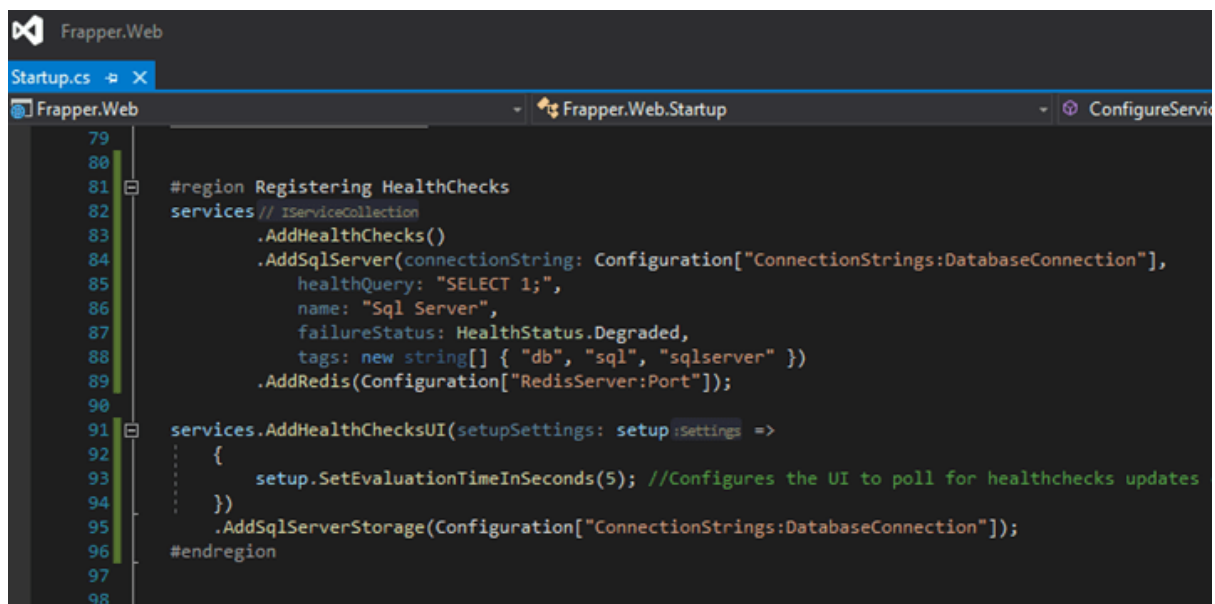
Resources

- LocalizationService.cs
- SharedResource.cs
 - SharedResource.de.resx
 - SharedResource.en.resx
 - SharedResource.hi.resx
 - SharedResource.ja.resx
 - SharedResource.mr.resx

Resource file



HealthChecks



HealthChecks

The screenshot shows the 'Health Checks UI' interface. The main heading is 'Health Checks Status'. On the right, it indicates 'Polling interval: 5 secs' and a 'Stop polling' button. The interface displays a table of health checks.

NAME	HEALTH	ON STATE FROM	LAST EXECUTION
HTTP-API-Basic	Healthy	2020-11-07T17:22:48.7571387+05:30	19/11/2020, 18:13:55

NAME	TAGS	HEALTH	DESCRIPTION	DURATION	DETAILS
Sql Server	db, sql	Healthy		00:00:00.0191241	
redis		Unhealthy	It was not possible to connect to the redis server(s). UnableToConnect on 0.0.24.236:6379/Interactive, Initializing/NotStarted, last: NONE, origin: BeginConnectAsync, outstanding: 0, last-read: 0s ago, last-write: 0s ago, keep-alive: 60s, state: Connecting, mgr: 10 of 10 available, last-heartbeat: never, global: 5s ago, v: 2.1.58.34321	00:00:10.5170746	

jQuery Datetime pickers & Bootstrap Datetime pickers

- jQuery Datetime picker

The screenshot shows the 'Frappier' application interface for 'Administration/Report/Export'. The 'Report' section is active, showing 'Export' options. The 'Report' dropdown is set to 'User Report' and 'ReportType' is 'Summary Report'. The 'Fromdate' is '2020-11-01' and 'Todate' is '2020-11-10'. A 'Download Report' button is visible. A jQuery Datetime picker calendar is open, showing the month of November 2020, with the 18th selected.

Copyright © 2014-2020 AdminLTE.io. All rights reserved. Frapper

- **Bootstrap Date Range Picker**

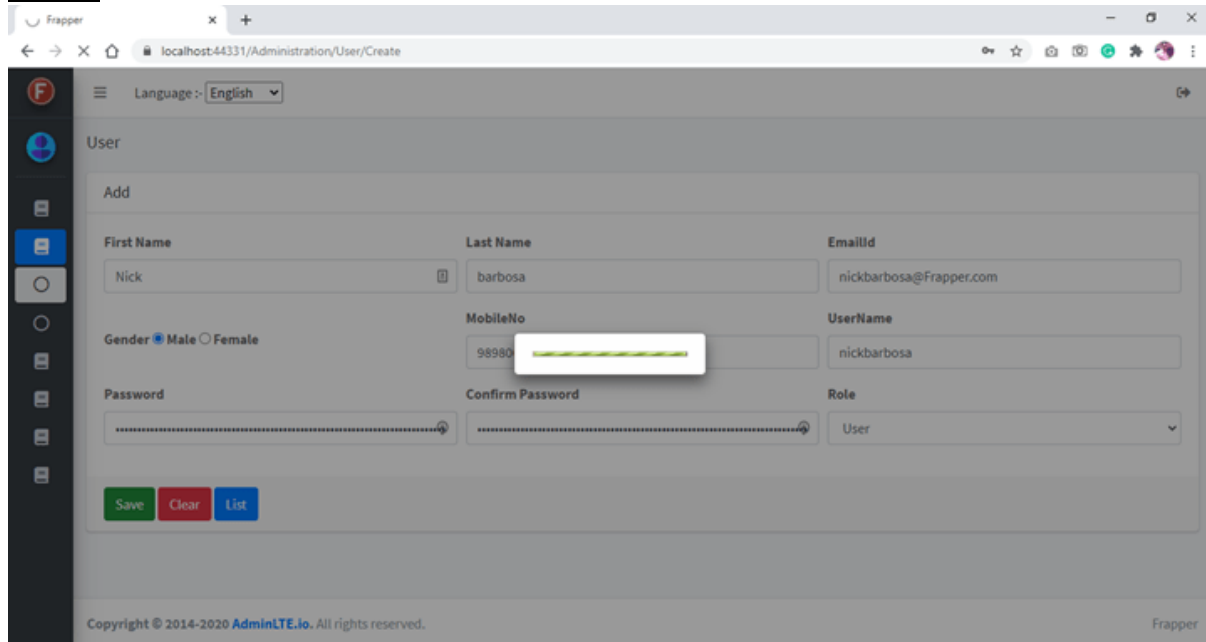
The screenshot shows the 'Add' form for creating a new notice. The form includes fields for 'NoticeTitle' (with a placeholder 'Enter FullName'), 'NoticeStart' (with a date '2020-11-18 19:23:24'), and 'NoticeEnd' (with a placeholder 'Enter Notice End Date'). Below these is a rich text editor for 'NoticeBody'. A date range picker is open, showing a calendar for November 2020 with the date '18' selected. The picker also shows the time '7:23 PM'. At the bottom of the form are 'Save', 'Clear', and 'List' buttons. The footer contains the copyright notice 'Copyright © 2014-2020 AdminLTE.io. All rights reserved.' and the name 'Frapper'.

Table Grid

The screenshot shows the 'Table Grid' view of the Frapper Notice management interface. It displays a table of notices with columns for 'NoticeTitle', 'NoticeStart', 'NoticeEnd', 'CreatedOn', 'Status', 'Edit', and 'Action'. The table contains two entries: 'Welcome to Frapper' and 'Notice for Users'. The 'Status' column shows 'Active' and 'InActive' respectively. The 'Action' column contains 'Edit' buttons and 'Active / InActive' status indicators. The interface also includes a search bar, a 'Show 10 entries' dropdown, and pagination controls at the bottom. The footer contains the copyright notice 'Copyright © 2014-2020 AdminLTE.io. All rights reserved.' and the name 'Frapper'.

NoticeTitle	NoticeStart	NoticeEnd	CreatedOn	Status	Edit	Action
Welcome to Frapper	2020-11-16 11:39:00	2021-11-20 10:46:00	2020-11-16 11:41:44	Active	Edit	Active / InActive
Notice for Users	2020-11-24 12:55:00	2020-11-26 12:55:00	2020-11-16 12:55:39	InActive	Edit	Active / InActive

Loader



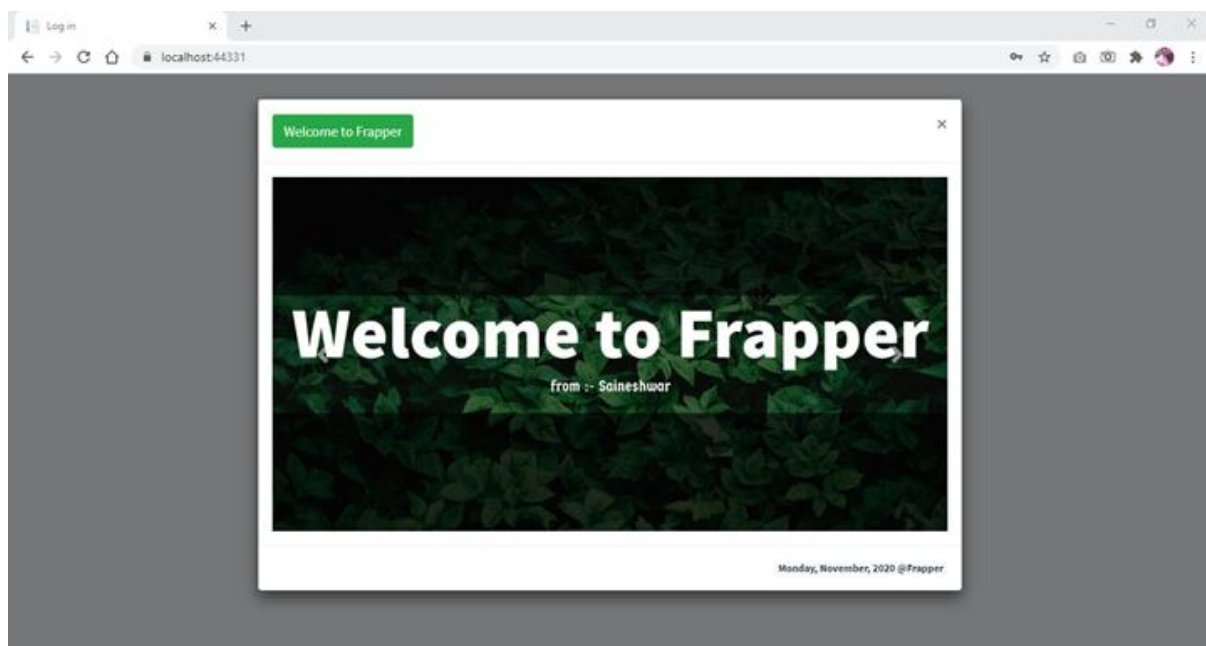
The screenshot shows a web browser window with the URL `localhost:44331/Administration/User/Create`. The page is titled "User" and has a language dropdown set to "English". The form is for creating a new user and includes the following fields:

- Add** (Section Header)
- First Name**: Nick
- Last Name**: barbosa
- EmailId**: nickbarbosa@Frappier.com
- Gender**: ☒ Male ☐ Female
- MobileNo**: 98980 (with a validation error message: "Please enter a valid mobile number")
- UserName**: nickbarbosa
- Password**: (masked with dots)
- Confirm Password**: (masked with dots)
- Role**: User (dropdown menu)

At the bottom of the form are three buttons: **Save** (green), **Clear** (red), and **List** (blue). The footer of the page reads: "Copyright © 2014-2020 AdminLTE.io. All rights reserved." and "Frappier".

Modal popups

We have used bootstrap modal popups and we have also rendered form in modal popup. And wrote whole validation using jQuery validation plugin.



Modal popups

The modal 'Add Customer' contains the following fields:

- ファーストネーム (First Name): Enter FirstName
- 苗字 (Last Name): Enter LastName
- 電子メールアドレス (Email Address): Enter EmailId
- 携帯電話番号 (Mobile Phone Number): Enter MobileNo
- 固定電話番号 (Landline Number): Enter LandlineNo
- 状態 (State): Enter State
- 市 (City): Enter City
- 通り (Street): Enter Street
- ピンコード (Pin Code): Enter Pincode

Each field has a red error message below it: 'Please enter [Field Name]'. A green 'Save' button is at the bottom.

Notification

We are using Toasts Notification in this project.



Notification Service

```

Frappier.Web - NotificationService.cs
NotificationService.cs
Frappier.Web
Frappier.Web.Messages.NotificationService
NotificationListKey

14
15 private readonly ITempDataDictionaryFactory _tempDataDictionaryFactory;
16 private readonly IHttpContextAccessor _httpContextAccessor;
17
18 public NotificationService(ITempDataDictionaryFactory tempDataDictionaryFactory, IHttpContextAccessor httpContextAccessor)
19 {
20     _tempDataDictionaryFactory = tempDataDictionaryFactory;
21     _httpContextAccessor = httpContextAccessor;
22 }
23
24 1 reference
25 public void Notification(string alertTitle, NotificationType type, string message, bool encode = true)
26 {
27     PrepareTempData(alertTitle, NotificationType.info, message, encode);
28 }
29
30 29 references
31 public void SuccessNotification(string alertTitle, string message, bool encode = true)
32 {
33     PrepareTempData(alertTitle, NotificationType.success, message, encode);
34 }
35
36 1 reference
37 public void WarningNotification(string alertTitle, string message, bool encode = true)
38 {
39     PrepareTempData(alertTitle, NotificationType.warning, message, encode);
40 }
41
42 1 reference
43 public void DangerNotification(string alertTitle, string message, bool encode = true)
44 {
45     PrepareTempData(alertTitle, NotificationType.danger, message, encode);
46 }
  
```

Notification Partial View

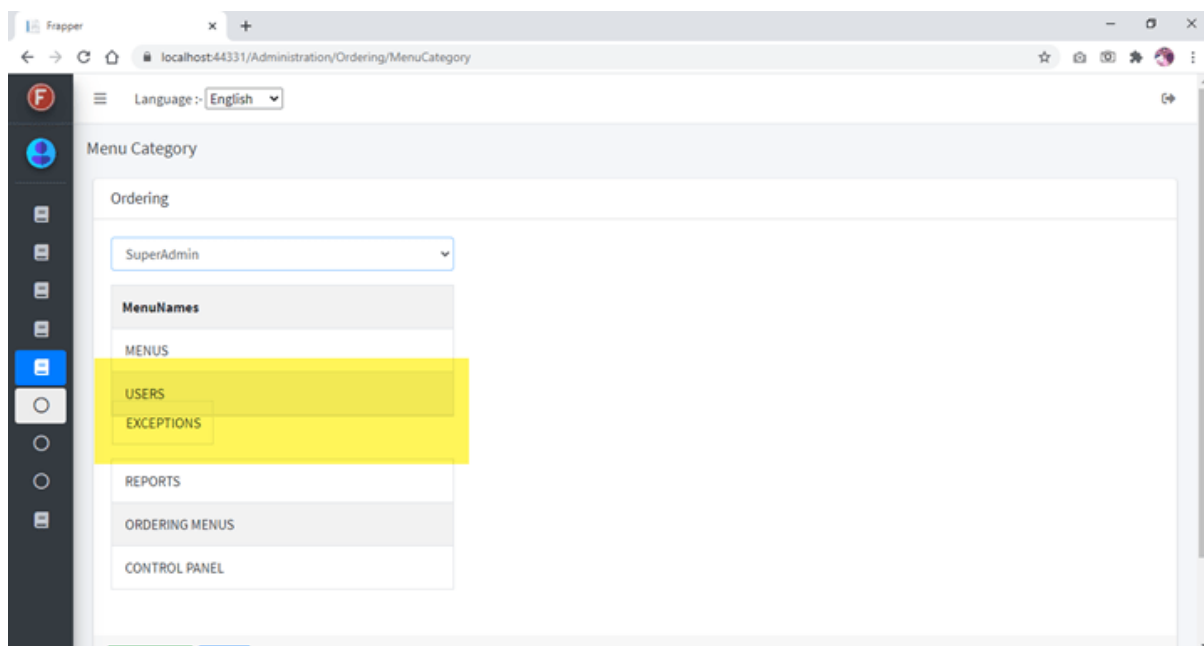
```

19 | if (note.Type == NotificationType.success)
20 | {
21 |     <script>
22 |         $(document).ready(function () {
23 |             displayToastsSuccessNotifications('@Html.Raw(note.AlertTitle)', '@Html.Raw(note.Message)');
24 |         });
25 |     </script>
26 | }
27 | if (note.Type == NotificationType.warning)
28 | {
29 |     <script>
30 |         $(document).ready(function () {
31 |             displayToastsWarningNotifications('@Html.Raw(note.AlertTitle)', '@Html.Raw(note.Message)');
32 |         });
33 |     </script>
34 | }
35 | if (note.Type == NotificationType.info)
36 | {
37 |     <script>
38 |         $(document).ready(function () {
39 |             displayToastsInformationNotifications('@Html.Raw(note.AlertTitle)', '@Html.Raw(note.Message)');
40 |         });
41 |     </script>
42 | }
43 |
44 | if (note.Type == NotificationType.danger)
45 | {
46 |     <script>
47 |         $(document).ready(function () {
48 |             displayToastsDangerNotifications('@Html.Raw(note.AlertTitle)', '@Html.Raw(note.Message)');
49 |         });
50 |     </script>

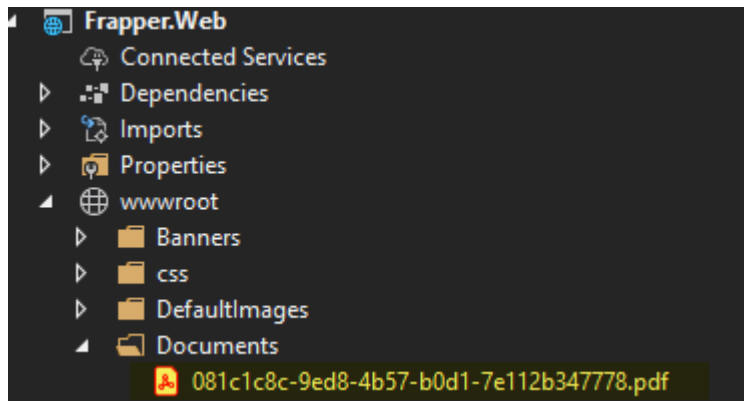
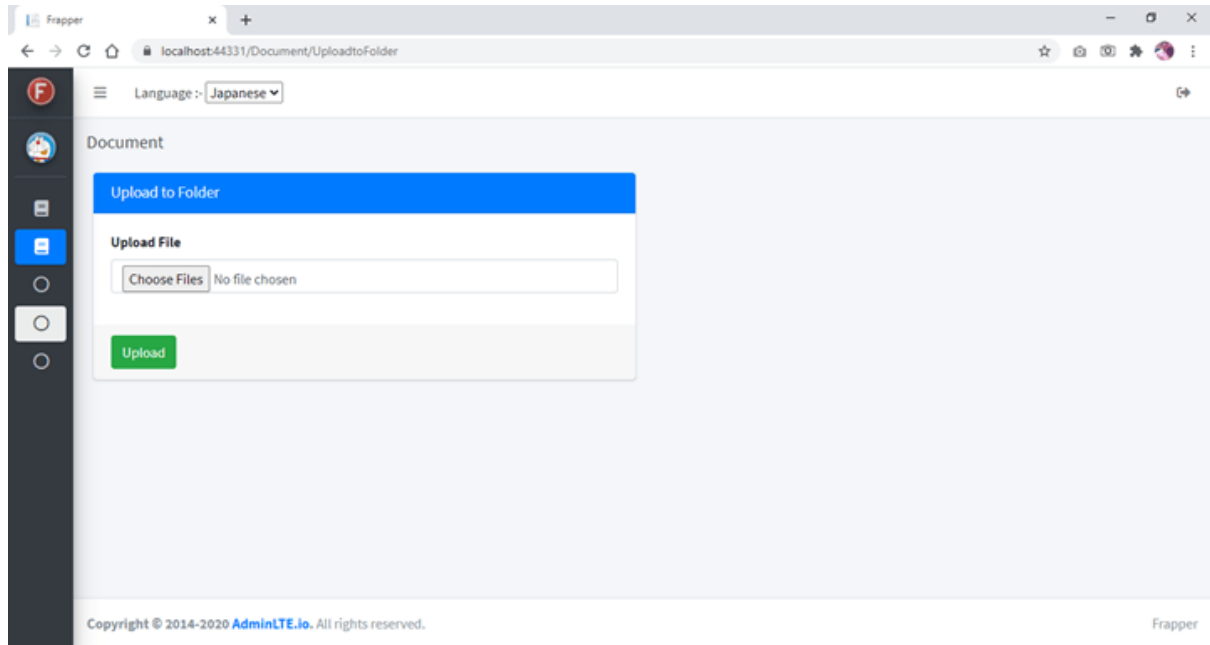
```

Ordering Menus

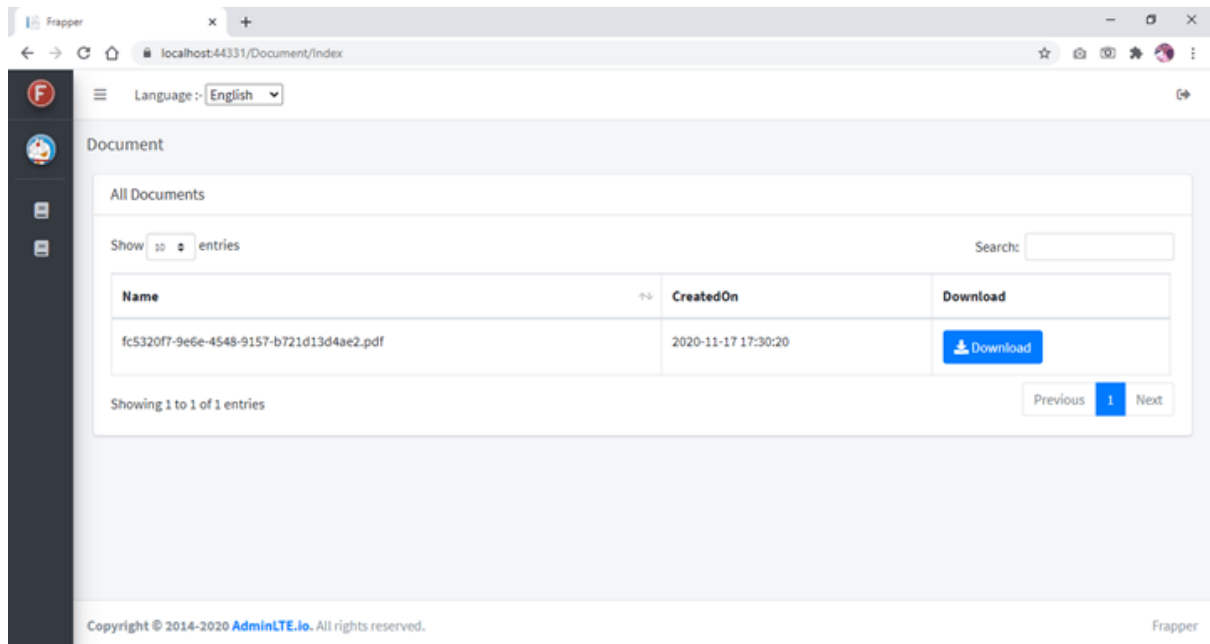
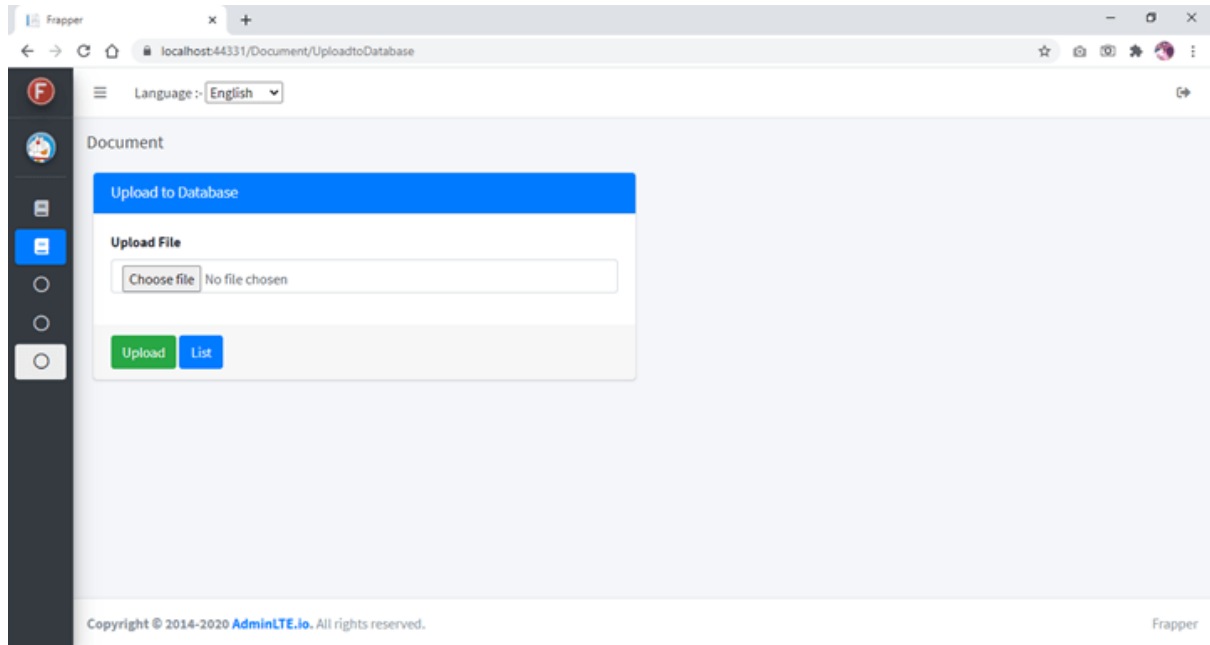
For ordering menus, we are using jQuery UI sortable.



Upload Files to store in a folder



Upload Files to store in Database



Reference : <https://github.com/saineshwar/Frapper>